SEM105 Lab 04: Eigenvectors & PCA (Group 8)

`pizza.csv` must be in the same folder as this notebook. Run every cell top to bottom (Run All) before taking screenshots.

Part I: Tutorial

In [1]:
```
import pandas as pd
dataset= pd.read_csv('pizza.csv')
dataset.head()
```

Out[1]:

	brand	id	mois	prot	fat	ash	sodium	carb	cal
0	A	14069	27.82	21.43	44.87	5.11	1.77	0.77	4.93
1	A	14053	28.49	21.26	43.89	5.34	1.79	1.02	4.84
2	A	14025	28.35	19.99	45.78	5.08	1.63	0.80	4.95
3	A	14016	30.55	20.15	43.13	4.79	1.61	1.38	4.74
4	A	14005	30.49	21.28	41.65	4.82	1.64	1.76	4.67

In [2]:
```
dataset.corr(numeric_only=True)
```

Out[2]:

	id	mois	prot	fat	ash	sodium	carb	c
id	1.000000	0.032595	-0.076485	-0.004860	-0.034072	-0.015156	0.014966	-0.0212
mois	0.032595	1.000000	0.360248	-0.171318	0.265556	-0.102279	-0.591802	-0.7644
prot	-0.076485	0.360248	1.000000	0.498002	0.823844	0.429130	-0.853542	0.0702
fat	-0.004860	-0.171318	0.498002	1.000000	0.791634	0.933325	-0.640238	0.7645
ash	-0.034072	0.265556	0.823844	0.791634	1.000000	0.808122	-0.898988	0.3264
sodium	-0.015156	-0.102279	0.429130	0.933325	0.808122	1.000000	-0.620176	0.6719
carb	0.014966	-0.591802	-0.853542	-0.640238	-0.898988	-0.620176	1.000000	-0.0234
cal	-0.021209	-0.764441	0.070258	0.764567	0.326468	0.671958	-0.023485	1.0000

In [3]:
```
import numpy as np
A= np.array([[3, 1], [1,3]])

l, v= np.linalg.eig(A)
```

```
print("Eigenvalues:", l)
print("Eigenvectors:", v)
```

```
Eigenvalues: [4.+0.j 2.+0.j]
Eigenvectors: [[ 0.70710678+0.j -0.70710678+0.j]
 [ 0.70710678+0.j  0.70710678+0.j]]
```

Newer NumPy versions return `np.linalg.eig` results as complex numbers. The `+0.j` parts are all zero, so the eigenvalues are 4 and 2 and the eigenvectors are the same as in the textbook.

In [4]:
```
#Dropping the brand name column before standardizing the data
df_num = dataset.drop(["brand", "id"], axis=1)

# Setting the brand name column as the target variable
target = dataset["brand"]

#Scaling the data (Step 1)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(df_num)
scaled_data = scaler.transform(df_num)

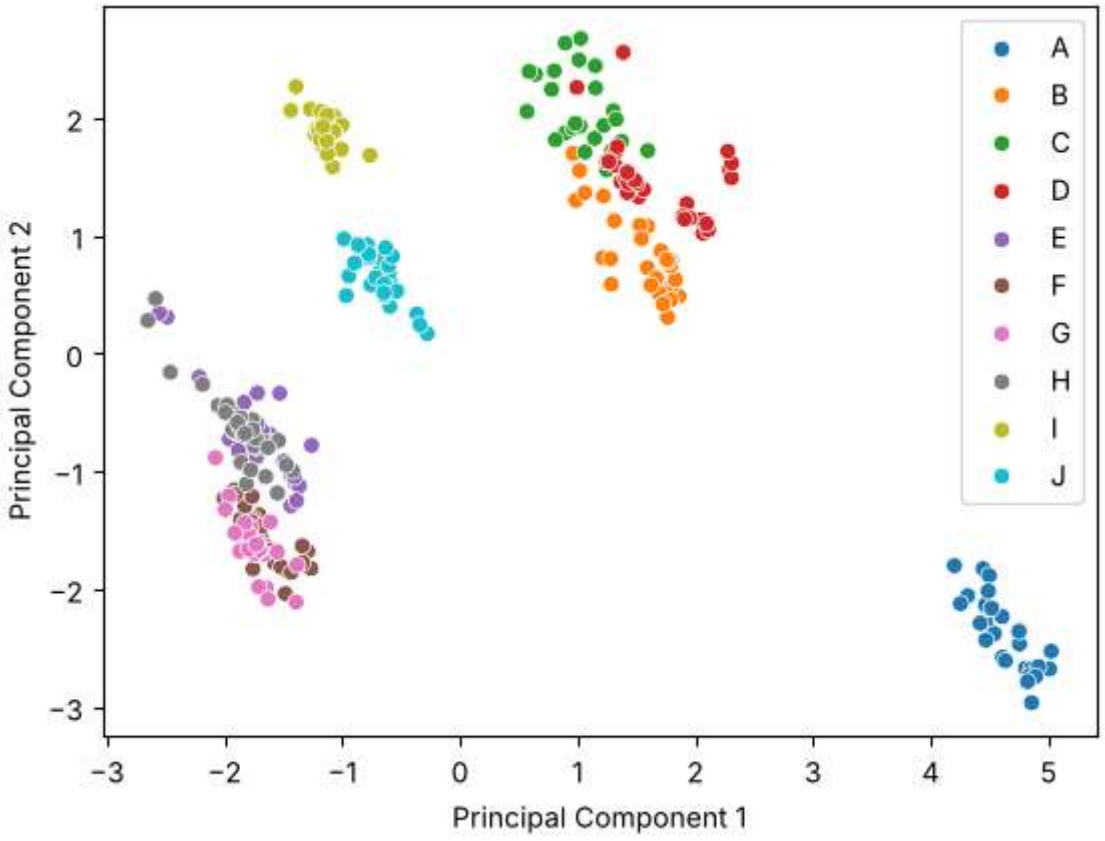
#Applying PCA to the scaled data
from sklearn.decomposition import PCA

#Reducing the dimensions to 2 components so that we can have a 2D visualization
pca = PCA(n_components = 2)
pca.fit(scaled_data)
#Applying to our scaled dataset
scaled_data_pca = pca.transform(scaled_data)
#Check the shape of the original dataset and the new dataset
print("The dimensions of the original dataset is: ", scaled_data.shape)
print("The dimensions of the dataset after performing PCA is: ", scaled_data_pca.sh
```

```
The dimensions of the original dataset is:  (300, 7)
The dimensions of the dataset after performing PCA is:  (300, 2)
```

In [5]:
```
#Plotting the principal components
import matplotlib.pyplot as plt
import seaborn as sns

sns.scatterplot(x=scaled_data_pca[:,0], y=scaled_data_pca[:,1], hue=target)
plt.legend(loc="best")
plt.gca().set_aspect("equal")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()
```



In [6]:
```
import pandas as pd

dataset = pd.read_csv('pizza.csv')

#Dropping the brand name column before standardizing the data
df_num = dataset.drop(["brand", "id"], axis=1)
# Setting the brand name column as the target variable
target = dataset['brand']

#Scaling the data (Step 1)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(df_num)
scaled_data = scaler.transform(df_num)

#Applying PCA to the scaled data
from sklearn.decomposition import PCA

#Setting the variance to 0.95
pca = PCA(n_components = 0.95)
pca.fit(scaled_data)
#Applying to our scaled dataset
scaled_data_pca = pca.transform(scaled_data)
#Check the shape of the original dataset and the new dataset
print("The dimensions of the original dataset are: ", scaled_data.shape)
print("The dimensions of the dataset after performing PCA is: ", scaled_data_pca.sh
```

```
The dimensions of the original dataset are:  (300, 7)
The dimensions of the dataset after performing PCA is:  (300, 3)
```

Part II: Exercises

(1) Manual PCA

$$X = \begin{bmatrix} 7 & 7 & 8 & 6 \\ 10 & 2 & 4 & 3 \\ 2 & 7 & 3 & 5 \\ 2 & 5 & 7 & 4 \\ 8 & 5 & 3 & 2 \end{bmatrix}$$

Standardize $z = \dfrac{x - \bar{x}}{s}$

$$\bar{x} = [5.8 \quad 5.2 \quad 5 \quad 4], \qquad s = [3.6332 \quad 2.0494 \quad 2.3452 \quad 1.5811]$$

$$Z = \begin{bmatrix} 0.3303 & 0.8783 & 1.2792 & 1.2649 \\ 1.1560 & -1.5614 & -0.4264 & -0.6325 \\ -1.0459 & 0.8783 & -0.8528 & 0.6325 \\ -1.0459 & -0.0976 & 0.8528 & 0 \\ 0.6055 & -0.0976 & -0.8528 & -1.2649 \end{bmatrix}$$

(a) Covariance matrix

$$C = \frac{1}{n-1} Z^T Z = \begin{bmatrix} 1 & -0.5977 & -0.1467 & -0.4352 \\ -0.5977 & 1 & 0.2601 & 0.6944 \\ -0.1467 & 0.2601 & 1 & 0.6068 \\ -0.4352 & 0.6944 & 0.6068 & 1 \end{bmatrix}$$

(b) Diagonalization, sort the order and solve $\det(\lambda I - C) = 0$:

$$\Lambda = \begin{bmatrix} 2.4129 & 0 & 0 & 0 \\ 0 & 0.9624 & 0 & 0 \\ 0 & 0 & 0.4325 & 0 \\ 0 & 0 & 0 & 0.1922 \end{bmatrix}$$

$$V = \begin{bmatrix} -0.4522 & 0.5586 & 0.6870 & -0.1076 \\ 0.5493 & -0.3069 & 0.5208 & -0.5769 \\ 0.3971 & 0.7382 & -0.3974 & -0.3734 \\ 0.5798 & 0.2209 & 0.3145 & 0.7184 \end{bmatrix}$$

Proportion of variance

$$\frac{\lambda_i}{\sum \lambda} = 0.6032,\ 0.2406,\ 0.1081,\ 0.0480$$

Cumulative: $60.3\%,\ 84.4\%,\ 95.2\%,\ 100\%$.

Choice of components: we keep **PC1 and PC2**. Together they explain $\frac{\lambda_1 + \lambda_2}{\sum \lambda} = \frac{2.4129 + 0.9624}{4} = 84.4\%$ of the variance, and two components let us plot the items in 2D. (Keeping three would explain 95.2%.)

(c) Transform

The eigenvectors were computed from the standardized data, so the feature matrix we multiply is the standardized Z.

$$Z\,V_2 = Z \begin{bmatrix} -0.4522 & 0.5586 \\ 0.5493 & -0.3069 \\ 0.3971 & 0.7382 \\ 0.5798 & 0.2209 \end{bmatrix} = \begin{bmatrix} 1.5744 & 1.1387 \\ -1.9164 & 0.6705 \\ 0.9835 & -1.3437 \\ 0.7580 & 0.0753 \\ -1.3994 & -0.5408 \end{bmatrix}$$

Eigenvector signs are arbitrary, so a column with all of its signs flipped is still a correct answer.

(2) PCA in Python

In [7]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

dataset = pd.DataFrame(
    {"item": ["item1", "item2", "item3", "item4", "item5"],
     "f1": [7, 10, 2, 2, 8],
     "f2": [7, 2, 7, 5, 5],
     "f3": [8, 4, 3, 7, 3],
     "f4": [6, 3, 5, 4, 2]})

# Dropping the label column before standardizing (like dropping "brand")
df_num = dataset.drop(["item"], axis=1)
target = dataset["item"]
dataset
```

Out[7]:

	item	f1	f2	f3	f4
0	item1	7	7	8	6
1	item2	10	2	4	3
2	item3	2	7	3	5
3	item4	2	5	7	4
4	item5	8	5	3	2

In [8]:
```
np.set_printoptions(precision=4, suppress=True)
```

```
X = df_num.values.astype(float)
n = X.shape[0]

Z = (X - X.mean(axis=0)) / X.std(axis=0, ddof=1)
print("Standardized data Z:\n", Z)

C = Z.T @ Z / (n - 1)
print("\nCovariance matrix C:\n", C)
```

```
Standardized data Z:
 [[ 0.3303  0.8783  1.2792  1.2649]
 [ 1.156  -1.5614 -0.4264 -0.6325]
 [-1.0459  0.8783 -0.8528  0.6325]
 [-1.0459 -0.0976  0.8528  0.    ]
 [ 0.6055 -0.0976 -0.8528 -1.2649]]

Covariance matrix C:
 [[ 1.     -0.5977 -0.1467 -0.4352]
 [-0.5977  1.      0.2601  0.6944]
 [-0.1467  0.2601  1.      0.6068]
 [-0.4352  0.6944  0.6068  1.    ]]
```

In [9]:
```
# Eigenvalue and vec , sort

eigvals, eigvecs = np.linalg.eigh(C)
order = np.argsort(eigvals)[::-1]
eigvals = eigvals[order]
eigvecs = eigvecs[:, order]

print("Diagonal eigenvalue matrix:\n", np.diag(eigvals))
print("\nEigenvector matrix (columns = PC1..PC4):\n", eigvecs)

ratios = eigvals / eigvals.sum()

print("\nProportion of variance:", ratios)
print("Cumulative:", np.cumsum(ratios))
```

```
Diagonal eigenvalue matrix:
 [[2.4129 0.     0.     0.    ]
 [0.     0.9624 0.     0.    ]
 [0.     0.     0.4325 0.    ]
 [0.     0.     0.     0.1922]]

Eigenvector matrix (columns = PC1..PC4):
 [[-0.4522  0.5586  0.687  -0.1076]
 [ 0.5493 -0.3069  0.5208 -0.5769]
 [ 0.3971  0.7382 -0.3974 -0.3734]
 [ 0.5798  0.2209  0.3145  0.7184]]

Proportion of variance: [0.6032 0.2406 0.1081 0.048 ]
Cumulative: [0.6032 0.8438 0.952  1.    ]
```

In [10]:
```
# keep 2 principal components
# set k =2 for later changes
k=2
```

```
new_data = Z @ eigvecs[:, :k]
print("Transformed data (Z x V2):\n", new_data)
```

```
Transformed data (Z x V2):
 [[ 1.5744  1.1387]
 [-1.9164  0.6705]
 [ 0.9835 -1.3437]
 [ 0.758   0.0753]
 [-1.3994 -0.5408]]
```

Scikit-learn PCA

`StandardScaler` divides by n instead of $n-1$, so the sklearn coordinates are $\sqrt{5/4} \approx 1.118$ times the manual ones. The components and explained variance ratios are the same as the manual calculation.

In [11]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

scaler = StandardScaler()
scaler.fit(df_num)
scaled_data = scaler.transform(df_num)

pca = PCA(n_components = 2)
pca.fit(scaled_data)
scaled_data_pca = pca.transform(scaled_data)

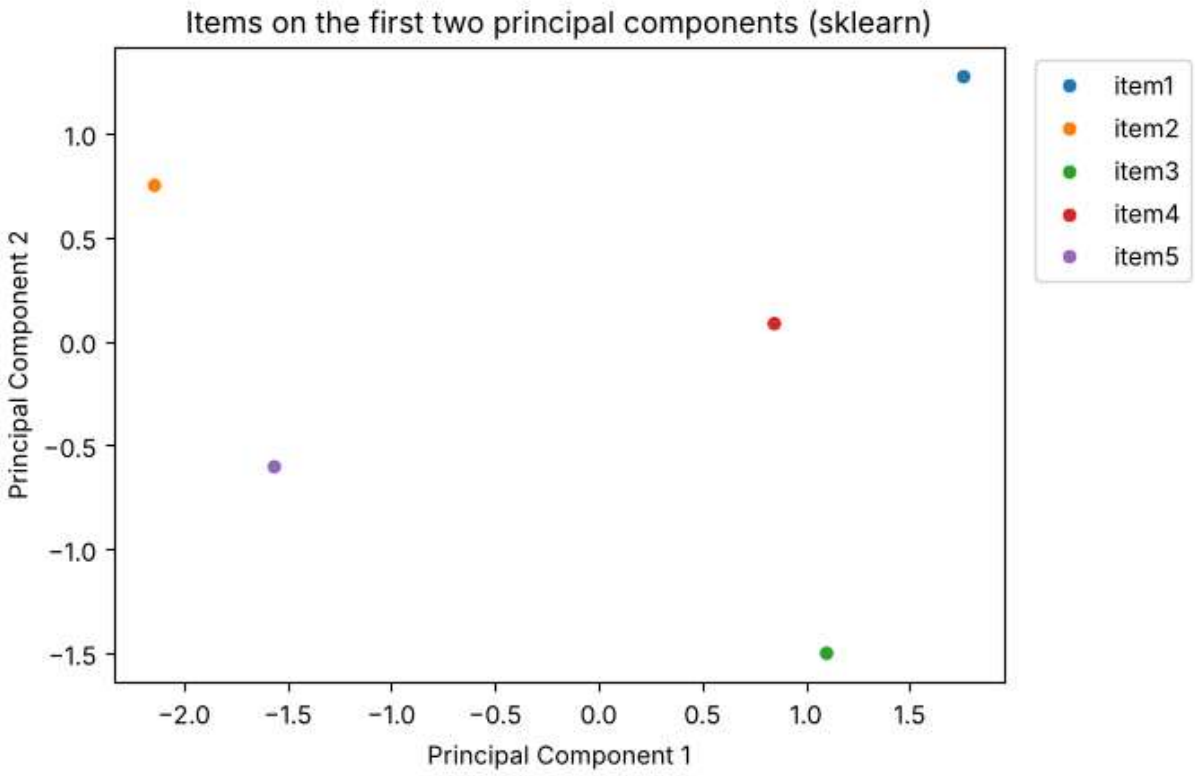
print("The dimensions of the original dataset are: ", scaled_data.shape)
print("The dimensions of the dataset after performing PCA is: ", scaled_data_pca.sh
print("\nsklearn components (rows = PCs):\n", pca.components_)
print("sklearn explained variance ratio:", pca.explained_variance_ratio_)
```

```
The dimensions of the original dataset are:  (5, 4)
The dimensions of the dataset after performing PCA is:  (5, 2)

sklearn components (rows = PCs):
 [[-0.4522  0.5493  0.3971  0.5798]
 [ 0.5586 -0.3069  0.7382  0.2209]]
sklearn explained variance ratio: [0.6032 0.2406]
```

In [12]:
```
#Plotting the principal components
import matplotlib.pyplot as plt
import seaborn as sns

sns.scatterplot(x=scaled_data_pca[:,0], y=scaled_data_pca[:,1], hue=target)
plt.legend(loc="upper left", bbox_to_anchor=(1.02, 1))
plt.gca().set_aspect("equal")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Items on the first two principal components (sklearn)")
plt.show()
```



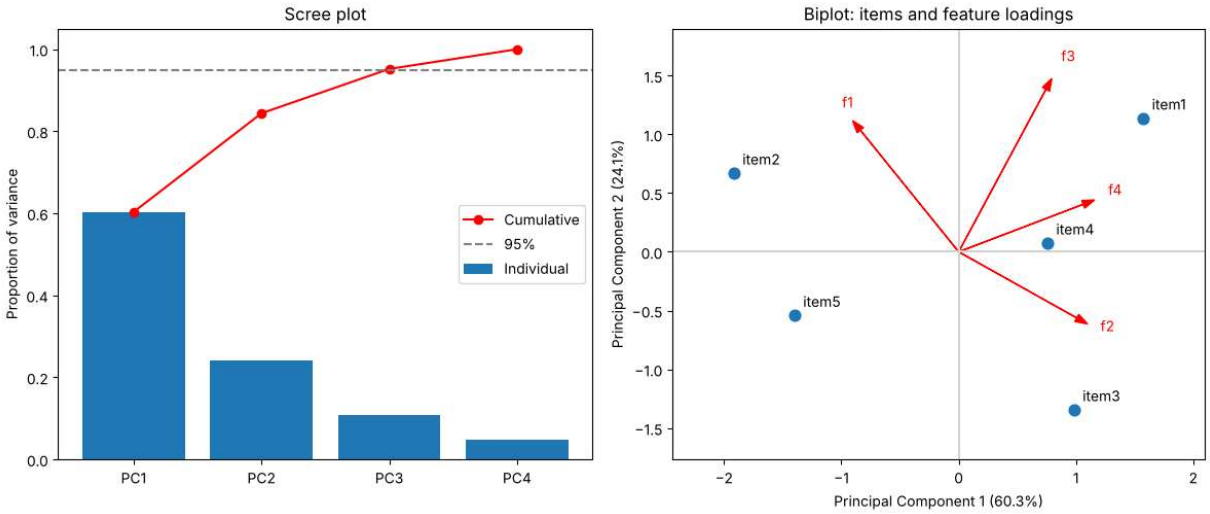
Graphical interpretation

In [13]:
```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Scree plot: how much variance each PC explains
pcs = ["PC1", "PC2", "PC3", "PC4"]
axes[0].bar(pcs, ratios, label="Individual")
axes[0].plot(pcs, np.cumsum(ratios), marker="o", color="red", label="Cumulative")
axes[0].axhline(0.95, linestyle="--", color="gray", label="95%")
axes[0].set_ylabel("Proportion of variance")
axes[0].set_title("Scree plot")
axes[0].legend(loc="center right")

# Biplot: items on PC1/PC2 (manual values) + arrows for each feature's loadings
ax = axes[1]
ax.scatter(new_data[:, 0], new_data[:, 1], s=60)
for i, name in enumerate(target):
    ax.annotate(name, (new_data[i, 0], new_data[i, 1]),
                textcoords="offset points", xytext=(6, 6))
for j, feat in enumerate(df_num.columns):
    ax.arrow(0, 0, 2 * eigvecs[j, 0], 2 * eigvecs[j, 1],
             color="red", head_width=0.07, length_includes_head=True)
    ax.text(2.2 * eigvecs[j, 0], 2.2 * eigvecs[j, 1], feat, color="red")
ax.axhline(0, color="lightgray")
ax.axvline(0, color="lightgray")
ax.set_xlabel("Principal Component 1 (%.1f%%)" % (100 * ratios[0]))
ax.set_ylabel("Principal Component 2 (%.1f%%)" % (100 * ratios[1]))
ax.set_title("Biplot: items and feature loadings")
ax.set_aspect("equal")
ax.margins(0.15)
```

```
plt.tight_layout()
plt.show()
```



Interpretation

- **Scree plot:** PC1 explains 60.3% and PC2 24.1%, so the two components we kept cover 84.4% of the variance. Three components would be needed to pass 95%.
- **PC1** has loadings -0.45 for f_1 and $+0.55, +0.40, +0.58$ for f_2, f_3, f_4, so it contrasts f_1 against the other three features. Items 2 and 5 are on the left because they are high in f_1 and low in f_2–f_4. Items 1, 3 and 4 are on the right.
- **PC2** is driven mostly by f_3 ($+0.74$), with some f_1 ($+0.56$). It separates item 1 (high f_3) from item 3 (low f_3 and f_1).
- Items 1 and 2 are farthest apart, so they are the most different items. Item 4 sits near the middle on PC2.
- The f_2 and f_4 arrows point in a similar direction because those features are positively correlated (0.69). The f_1 arrow points away from f_2 because they are negatively correlated (−0.60).